

Custom CPQ on Salesforce

Solution Design

DOCUMENT CONTROL

DOCUMENT ID	LP-REVTECH-CPQ-SD-001
VERSION	2.2
STATUS	For review
AUTHOR	Bruno Romano
DATE	8 September 2026
FOUNDATIONS	Nebula Logger (unlocked package) · Trigger Actions Framework
AUDIENCE	RevTech engineering · RevOps · Platform engineering
COMPANION ARTEFACT	Apex prototype and test suite (repository)
IMPLEMENTATION	Built and deployed · 66 Apex tests passing · exercised over HTTP from Postman
SCOPE	Catalog exposure, price-floor enforcement, commit to Opportunity, Closed Won entitlement automation, logging

ABSTRACT

This document specifies the Salesforce-side layer behind the CPQ experience being built on Luxury Presence's internal React/NestJS platform: the data model, the externally-callable integration surface, its security posture, the Apex automation that produces durable entitlements at Closed Won, and logging. The client application is out of scope.

The design has since been built, deployed, and called over HTTP by a least-privilege integration user. Several things that were reasonable on paper proved wrong in the org; those are recorded as findings in Appendix A rather than edited away. Decisions that are not self-evident are numbered **D-n**.

CONTENTS

1 Scope and the governing constraint	3
2 Assumptions, exclusions and open questions	3
2.1 Assumptions	3
2.2 Deliberately out of scope	4
2.3 Open questions	4
3 Business rules that become code	4
3.1 Catalog	4
3.2 Configuration	5
3.3 Pricing	5
3.4 Contract	5
3.5 Commit	5
3.6 Entitlements	5
4 Data model	6
4.1 Entity relationships	6
4.2 CPQ_Catalog_Item__c — everything sellable	7
4.3 CPQ_Feature__c — the capability catalog	7
4.4 CPQ_Commit__c — one row per commit attempt	8
4.5 Remaining objects	8
4.6 Why Product2 exists alongside a custom catalog	8
5 Entitlement aggregation	9
5.1 Worked examples	9
5.2 The algorithm	9
6 Integration surface	10
6.1 Why custom Apex REST	10
6.2 Endpoints	10
6.3 Status codes	11
6.4 Authentication and the rep's identity	12
6.5 Keeping data in sync	12
7 Security and observability	13
7.1 Enforcement	13
7.2 Least privilege, as actually established	13
7.3 Logging	13
8 Closed Won automation	14
9 Deferred scope	15
10 Test strategy	16
10.1 Acceptance criteria as test methods	16
10.2 The layer Apex tests cannot reach	16
10.3 Running it	17
Appendix A — What building it changed	17

1 Scope and the governing constraint

Reps configure deals in the React/NestJS platform. Salesforce is the catalog of record, the authority that enforces commercial rules, and the origin of durable entitlements. No one opens a Salesforce screen in this workflow.

That produces a single constraint everything else follows from:

Salesforce cannot trust the caller. Every rule below has to hold when the request comes from a script, a Postman tab, or a bug in the platform — not only from the intended UI.

The practical consequences: validation is re-run inside commit and a prior `validate` call is never trusted; the price floor lives in Apex and is never returned as advisory data the client is expected to enforce; and the API surface is typed and versioned, because a client I don't control will build against it and I can't redeploy their code.

D-1 · DESIGN DECISION — WHERE THE RULES LIVE

The commercial rules run inside the endpoint, not in the client and not in configuration the client reads. A floor returned to the platform as data is a floor the platform can ignore. This is the reason the integration surface is custom Apex REST rather than the standard sObject API, and the reason `commit` re-validates rather than trusting a prior `validate`.

2 Assumptions, exclusions and open questions

2.1 Assumptions

#	ASSUMPTION	WHY IT MATTERS
A-1	Add-on list prices, floors and charge types are invented. Design Refresh \$750 / floor \$600 / one-time; Competitor Analysis \$150 / floor \$120 / recurring; Contacts (500) \$100 / floor \$80 / recurring.	The brief names the three add-ons but not their commercials. These live in catalog records — RevOps changes them without a deployment.
A-2	Plan prices are monthly recurring; setup fees are one-time; billing frequency is monthly.	Drives the recurring vs. one-time split that billing and entitlements both key off.
A-3	USD only.	Multi-currency would change the catalog to a per-currency price row. Out of scope, but the model isn't blocked from it.
A-4	The Opportunity has an Account before commit.	Entitlements are held against a customer. An Opportunity without an Account cannot produce them, so commit rejects it.
A-5	Permitted contract terms are 12/24/36 months, held as configuration rather than a hard-coded list.	Sales will change this.

#	ASSUMPTION	WHY IT MATTERS
A-6	The setup fee is discountable to \$0 but never above the catalog amount.	Floor-style rule in the other direction. Flagged as Q-3.
A-7	One active Subscription per Account at a time.	Concurrent subscriptions are a renewals concern, deliberately out of scope.
A-8	The platform holds its own credentials and passes the rep's identity in the request body. Reps never hold Salesforce credentials.	Without this, no discount is attributable to a person. See §6.4.

2.2 Deliberately out of scope

No UI, no Opportunity creation, no live callouts to billing or feature gating, no approval workflow, and no use of `Quote / QuoteLineItem` — `OpportunityLineItem` is the source of truth for what is being sold, since contracts are generated in PandaDoc. Renewals, amendments, co-terming, proration and cancellation are a later phase; the model is shaped not to preclude them (§9).

2.3 Open questions

#	QUESTION	WHY IT'S OPEN
Q-1	Is uplift above list price permitted, or is there a ceiling as well as a floor?	Currently unbounded. A ceiling is one rule in the same validator.
Q-2	Confirm add-on prices, floors and charge types (A-1).	Data change only, no code impact.
Q-3	Can any rep waive the setup fee to \$0, or does the waiver need its own floor or an approval?	Currently freely waivable. This is the discount lever most likely to be abused once the plan floor is enforced.
Q-4	When a contract term ends, who deactivates entitlements — a scheduled job, or the gating service reading the end date?	The model supports both. The operational owner is undecided.
Q-5	The integration user needs a full Salesforce licence, not the cheaper Integration licence (F-1). Is that acceptable, or should the API avoid <code>OpportunityLineItem</code> entirely?	A licensing cost, discovered in the build. Avoiding it would mean not writing Opportunity Products, which changes what the deal record means.

3 Business rules that become code

Only the rules a test asserts against. Each maps to a test method in §10.

3.1 Catalog

R-1 — Catalog items carry a stable, human-readable code (`PLAN_BRAND` , `ADDON_CONTACTS_500`). Downstream systems key on the code, never on the Salesforce Id.

R-2 — Only items active on the configuration's effective date may be quoted.

R-3 — Changing a catalog price or floor never retroactively alters committed lines or existing entitlements.

3.2 Configuration

- R-4 — Exactly one plan. Zero or multiple is invalid.
- R-5 — An add-on appears at most once; stacking is expressed as quantity, not repeated lines. Quantity is a positive integer.
- R-6 — Add-ons that duplicate a plan-included feature are valid in both directions, by design.

3.3 Pricing

- R-7 — Negotiated plan price $\geq$ that plan's floor. Inclusive: exactly at floor passes.
- R-8 — Each add-on has its own floor; a negotiated price below it fails.
- R-9 — Setup fee may be discounted to \$0 but may not exceed the catalog amount.
- R-10 — Prices are USD, non-negative, two decimal places.
- R-11 — Validation returns **every** violation in one response. A rep fixing three lines should make one round trip.

3.4 Contract

- R-12 — Effective date required; term must be one of the configured permitted values.

3.5 Commit

- R-13 — Commit re-runs full validation server-side. A prior successful validate is never trusted.
- R-14 — Commit is atomic. All lines and the Opportunity term fields, or nothing.
- R-15 — Commit is refused if the target Opportunity is already Closed.
- R-16 — Re-committing **replaces** the lines this API previously wrote, and leaves lines added by other means untouched.
- R-17 — Commit is idempotent by key: the same request twice yields one set of lines and returns the original result.

3.6 Entitlements

- R-18 — Entitlement quantity per feature is the sum of the plan-included quantity and matching add-on quantities. Binary features do not accumulate; metered features do.
 - R-19 — Only recurring, feature-bearing items produce entitlements. The setup fee and Design Refresh are billable and grant nothing.
 - R-20 — Entitlement creation is idempotent and bulk-safe. A 200-record stage update, or an already-won Opportunity re-saved, creates no duplicates.
 - R-21 — Entitlements are durable: superseded, never silently overwritten.
-

4 Data model

4.1 Entity relationships

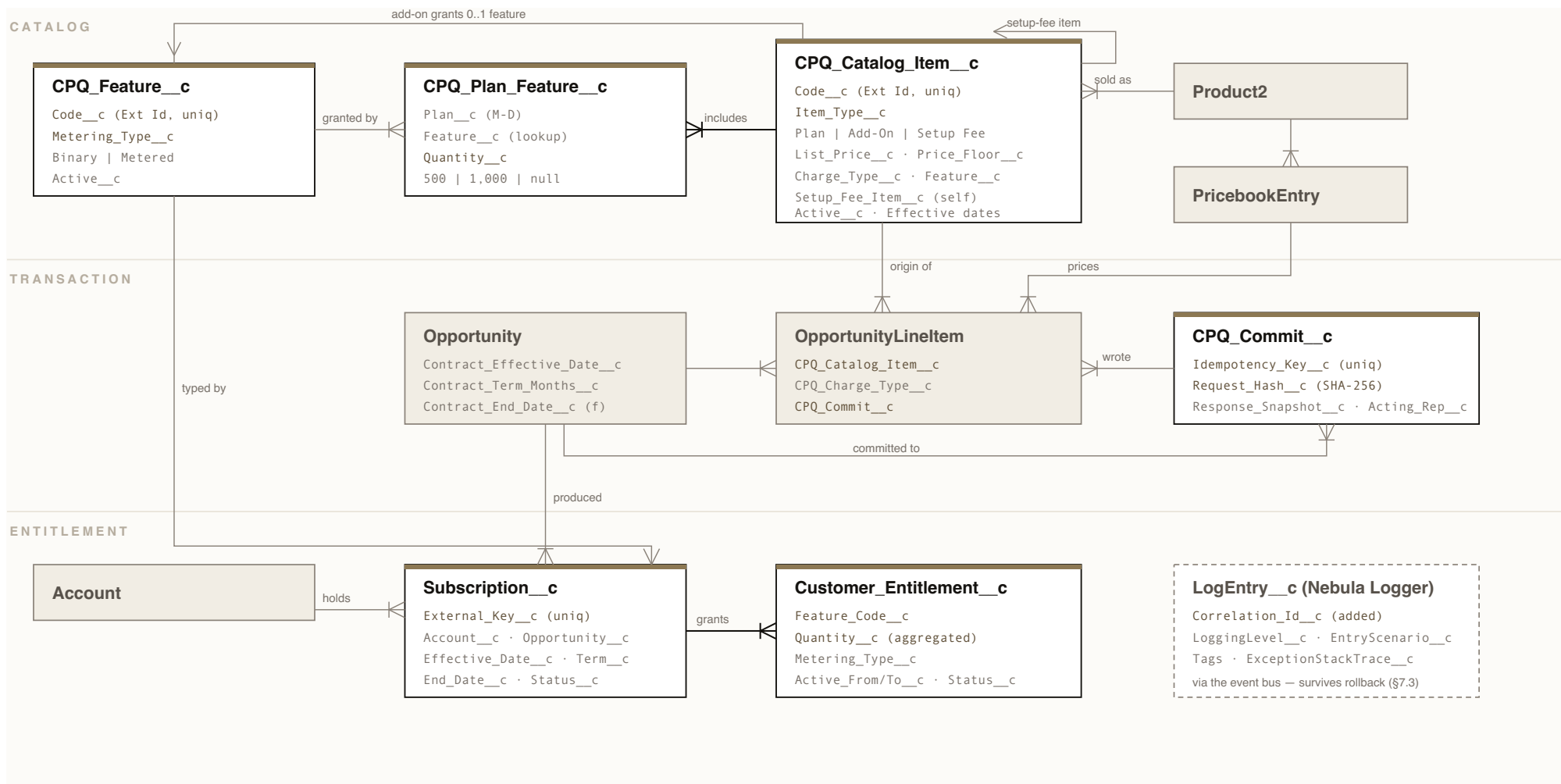


FIGURE 1 Entity-relationship diagram. Three bands: the catalog, the transaction layer that records what was sold, and the entitlements a won deal produces. Custom objects built by this project carry a brass top rule; standard Salesforce objects are shaded. **LogEntry__c** is Nebula Logger's, shown dashed because it holds no relationships by design (§7.3).

4.2 CPQ_Catalog_Item__c — everything sellable

Plans, add-ons and setup fees share a shape: a code, a list price, a floor, an active window and a `Product2`.

FIELD	TYPE	NOTE
<code>Code__c</code>	Text, External Id, Unique	<code>PLAN_BRAND</code> , <code>ADDON_CONTACTS_500</code> , <code>SETUP_BRAND</code>
<code>Item_Type__c</code>	Picklist	<code>Plan</code> / <code>Add-On</code> / <code>Setup Fee</code>
<code>List_Price__c</code> , <code>Price_Floor__c</code>	Currency(16,2)	
<code>Charge_Type__c</code>	Picklist	<code>Recurring Monthly</code> / <code>One-Time</code>
<code>Product__c</code>	Lookup(Product2)	Required. See §4.6
<code>Setup_Fee_Item__c</code>	Lookup(self)	Plans only → the <code>Setup Fee</code> item
<code>Feature__c</code>	Lookup(<code>CPQ_Feature__c</code>)	Add-ons only. Null for Design Refresh
<code>Feature_Quantity_Per_Unit__c</code>	Number	Contacts (500) → <code>500</code>
<code>Active__c</code> , <code>Effective_Start__c</code> , <code>Effective_End__c</code>	Checkbox, Date, Date	<code>R-2</code>

D-2 · DESIGN DECISION — SETUP FEES ARE CATALOG ITEMS

Modelling setup fees as catalog items rather than as a number on the plan means every committed line is uniformly one catalog item → one `Product2` → one `OpportunityLineItem`. There is no second code path for the setup fee, and RevOps can reprice one without touching plan records.

4.3 CPQ_Feature__c — the capability catalog

Kept separate from the product catalog. Products are what we *sell*; features are what a customer is *entitled to*. Competitor Analysis is both, and conflating them is what makes the overlap case hard.

FIELD	TYPE	NOTE
<code>Code__c</code>	Text, External Id, Unique	<code>FEAT_CONTACTS</code>
<code>Metering_Type__c</code>	Picklist	<code>Binary</code> / <code>Metered</code> — decides whether quantities accumulate
<code>Active__c</code>	Checkbox	

`CPQ_Plan_Feature__c` is the junction: master-detail to the plan, lookup to the feature, plus `Quantity__c`. Plan inclusion carries a quantity (All In → `FEAT_CONTACTS` × 1,000), so it cannot be a checkbox.

4.4 CPQ_Commit__c — one row per commit attempt

`Idempotency_Key__c` is a **unique External Id**, which is what makes `R-17` real: uniqueness is enforced by the database, so two concurrent identical requests cannot both win.

`Request_Hash__c` is a SHA-256 of the configuration being sold — plan, prices, add-ons, term — excluding the fields that legitimately differ between a request and its retry.

D-3 · DESIGN DECISION — IDEMPOTENCY IS ENFORCED BY THE DATABASE

Querying for an existing key and then inserting would pass every test in the repository and still lose a race in production. A unique External Id makes the second writer fail rather than duplicate.

The request hash is what distinguishes "same key, same request" from "same key, different request". Without it the `409` in §6.3 cannot be implemented: you would have to either replay a result for a payload the caller never sent, or silently overwrite. Neither is safe.

4.5 Remaining objects

OpportunityLineItem — three custom fields: `CPQ_Catalog_Item__c` (durable link back to the catalog, so Closed Won never re-derives anything by name matching), `CPQ_Charge_Type__c`, and `CPQ_Commit__c`. That last one is how `R-16` works: replacing a configuration deletes only lines stamped by a prior commit of this API.

Opportunity — `Contract_Effective_Date__c`, `Contract_Term_Months__c`, `Contract_End_Date__c` (formula).

Subscription__c — Account, Opportunity, plan, effective date, term, end date, status. `External_Key__c` (unique External Id, set to the Opportunity Id) makes the Closed Won automation idempotent under `upsert`.

Customer_Entitlement__c — named to avoid colliding with the standard Service Cloud `Entitlement` object. Master-detail to Subscription, lookup to Feature, plus `Feature_Code__c` (denormalised so downstream never joins), `Quantity__c` (already aggregated), `Metering_Type__c`, `Active_From__c` / `Active_To__c`, and `Status__c`.

4.6 Why Product2 exists alongside a custom catalog

`OpportunityLineItem` cannot be inserted without a `PricebookEntryId`. That is a platform constraint, not a preference. So each sellable catalog item maps to a `Product2` with a standard `PricebookEntry`.

D-4 · DESIGN DECISION — DIVISION OF RESPONSIBILITY

`Product2 / PricebookEntry` is the commercial record the platform requires. `CPQ_Catalog_Item__c` is the source of truth for the CPQ rules — floors, included features, charge type, effective windows — none of which `Product2` can express.

Putting floors in a `PricebookEntry` custom field would have saved an object, but it scatters the rule set across a standard object shared with every other sales process in the org. The custom catalog keeps the rules in one place that RevOps owns.

This decision has a cost that only appeared in the build: because the API writes `OpportunityLineItem`, the integration principal needs access to `Opportunity`, `Product2` and `Pricebook2`, and that rules out the cheap Salesforce Integration licence entirely (`F-1`).

5 Entitlement aggregation

5.1 Worked examples

The case the brief calls out, plus one that proves the logic is not hard-coded to two sources.

CASE 1 — BRAND (\$595/MO, INCLUDES CONTACT PACK ×1) + CONTACTS (500) ×1

FEATURE	FROM PLAN	FROM ADD-ONS	ENTITLEMENT
Website hosting	enabled	—	enabled
Listing feeds	enabled	—	enabled
Contacts	500	500	1,000

CASE 2 — ALL IN (\$2,495/MO, INCLUDES CONTACT PACK ×2) + CONTACTS (500) ×3 + COMPETITOR ANALYSIS ×1 + DESIGN REFRESH ×1

FEATURE	FROM PLAN	FROM ADD-ONS	ENTITLEMENT
Website hosting	enabled	—	enabled
Listing feeds	enabled	—	enabled
Competitor analysis	enabled	enabled	enabled — once
Contacts	1,000	1,500	2,500
Design Refresh	—	—	none

Both cases run in `scripts/demo.sh` against a real org, so the table above is output rather than intention.

D-5 · DESIGN DECISION — BINARY VS. METERED IS A FIELD, NOT AN IF

`CPQ_Feature__c.Metering_Type__c` decides whether the reducer sums or ORs. Competitor Analysis arriving from both the plan and an add-on resolves to enabled once because it is Binary; contacts accumulate because they are Metered.

Neither source is privileged — the plan is not a base that add-ons top up. Both contribute through the same path, which is what makes the overlap work in both directions as R-6 requires.

Design Refresh produces nothing because its `Feature__c` is null. There is no `if (code == 'ADDON_DESIGN_REFRESH')` anywhere in the codebase; the absence of the link is the whole mechanism, so the next non-entitling add-on needs no deployment.

5.2 The algorithm

1. Query committed `OpportunityLineItem`s for the won Opportunities where `CPQ_Catalog_Item__c != null`.
2. Skip `Setup Fee` and `One-Time` items (R-19).

3. Plan lines → their `CPQ_Plan_Feature__c` rows, contributing `Quantity__c`.
4. Add-on lines → their catalog item's `Feature__c`, contributing `Feature_Quantity_Per_Unit__c × OLI.Quantity`.
5. Reduce into `Map<Id oppId, Map<Id featureId, Decimal>>`, summing where metered and taking presence where binary.
6. Upsert one `Subscription__c` per Opportunity by external key, then insert its entitlements.

Two SOQL queries and two DML statements regardless of batch size.

One qualification, because it is the kind of claim that quietly stops being true. Regenerating over a subscription that already exists has to find the previous entitlements and supersede them, which costs one more query and one more DML. That path is entered only when the upsert reports the subscription already existed — so the two-and-two holds for every normal close, and the extra pair is paid only when there is genuinely prior history to preserve.

6 Integration surface

6.1 Why custom Apex REST

OPTION	VERDICT
Custom Apex REST (<code>@RestResource</code>)	Chosen. Typed request/response shapes, explicit status codes, an explicitly versioned URI, and — the deciding factor — the business rules run inside the endpoint where the client cannot route around them.
Standard sObject REST + Composite	Rejected. The platform would assemble line items itself, which means pricing rules live in the client. That is precisely the failure mode this project exists to prevent.
Invocable Apex	Rejected. Designed for Flow. Calling it externally gives an awkward wrapper, weaker typing and no natural place for HTTP semantics.
GraphQL / UI API	Rejected. No hook for server-side commercial rules.

6.2 Endpoints

```
GET /services/apexrest/cpq/v1/catalog?effectiveDate=2026-09-08
POST /services/apexrest/cpq/v1/configurations/validate
POST /services/apexrest/cpq/v1/opportunities/{opportunityId}/commit
```

GET /catalog returns plans (list price, floor, setup fee, included features and quantities) and add-ons (list price, floor, charge type, mapped feature), filtered to items active on the requested date. Every item carries its code so the platform never stores Salesforce Ids. The response includes `catalogVersion` — the max `SystemModstamp` across catalog objects — so the platform can cache and cheaply detect staleness.

POST /validate takes a proposed configuration and returns:

```
{
  "correlationId": "b3f1...",
  "valid": false,
  "violations": [
    { "code": "PLAN_BELOW_FLOOR", "line": "PLAN_BRAND",
      "submitted": 425.00, "permitted": 450.00,
      "message": "Brand plan price is below the $450.00 floor." },
    { "code": "ADDON_BELOW_FLOOR", "line": "ADDON_CONTACTS_500",
      "submitted": 70.00, "permitted": 80.00,
      "message": "Contacts (500) price is below the $80.00 floor." }
  ],
  "totals": null
}
```

Structured, not prose — the platform renders its own field-level errors from `line` and `code`. All violations in one response (`R-11`). No DML, no side effects, safe to call on every keystroke.

POST /commit takes the finalised configuration plus an `idempotencyKey`, re-validates from scratch, and writes the plan line, the setup-fee line, the add-on lines and the Opportunity contract fields inside one savepoint. It returns the Opportunity Id, the created line Ids, computed totals and the correlation ID.

6.3 Status codes

STATUS	MEANING
200	Success. Including a <code>validate</code> call that returns <code>valid: false</code>
422	Commit refused on business-rule violations — same payload shape as <code>validate</code>
400	Malformed request: bad JSON, missing field, wrong type, unparseable Id
404	Opportunity not found or not accessible — deliberately indistinguishable
409	Opportunity already Closed, or an idempotency key replayed with a different payload
403	The integration principal lacks required object or field access
500	Unexpected — typed error and correlation ID to the caller, stack trace to the log only

D-6 · DESIGN DECISION — THE 200/422 ASYMMETRY

`validate` asks *"is this valid?"*, and answering "no" is a successful call, so it is `200` with `valid: false`.
`commit` asks *"write this"*, and refusing is a failed action, so it is `422`.

This is stated explicitly because a client developer will otherwise guess, and guess differently.

Violation codes are a closed, documented set, enumerated once in `CpqConstants` and listed in the repository. The platform branches on the code and renders its own localised message rather than parsing prose. Codes are additive across versions; removing or repurposing one is a breaking change and earns `/v2`.

6.4 Authentication and the rep's identity

Pattern: a Connected App using the **OAuth 2.0 client credentials flow**, server-to-server, no user interaction, no refresh-token lifecycle for the platform to manage. It runs as a named integration user, `svc.revtech.cpq`, holding one permission set and nothing else.

The Connected App's *Run As* user and its consumer secret are the only two pieces of this design that cannot be deployed as metadata; both are set once in Setup. Everything else — the user, the permission set, the app itself — is in the repository.

A second permission set, `CPQ_Catalog_Admin`, carries RevOps' write access to the catalog.

D-7 · DESIGN DECISION — SPLIT THE SELLING PRINCIPAL FROM THE REPRICING PRINCIPAL

The principal that *sells* against the floor is not the principal that can *change* the floor. A compromised integration credential therefore cannot quietly reprice the catalog and then sell under it.

The rep is not the authenticated principal. Salesforce sees one integration user for every rep in the company. So the platform passes `actingRepEmail` in the request body; Salesforce resolves it to an active `User`, rejects it if it does not resolve, and stamps it on `CPQ_Commit__c` and every log entry.

D-8 · DESIGN DECISION — CLIENT-SUPPLIED IDENTITY IS ATTRIBUTION, NOT AUTHORISATION

Access is governed entirely by the integration user's permissions and sharing. Treating a client-supplied identity as an authorisation claim would let anyone holding the integration credentials act as anyone.

But without capturing it, no discount is traceable to a person, and "who sold this at the floor?" becomes unanswerable — which is half the reason for enforcing floors at all.

6.5 Keeping data in sync

Catalog (Salesforce → platform): pull with caching, keyed on `catalogVersion`. The catalog changes a few times a quarter and staleness is low-risk — a stale floor in the client's UI is caught by the server-side check at commit, which is exactly why that check exists.

Entitlements (Salesforce → downstream): the model is shaped so this becomes a *publish*, not a redesign. Because entitlements are already resolved rows with a code, a quantity and an active window, emitting a platform event on insert is a small addition. Billing is better served by Change Data Capture on `OpportunityLineItem` and `Subscription__c`.

D-9 · DESIGN DECISION — NO CALLOUTS FROM THE CLOSED WON TRIGGER

A callout would make the sales rep's stage change depend on the availability of two other systems. If billing is down, the deal does not close. Events decouple that, and a lost event is recoverable by replay in a way that a failed synchronous callout inside a rolled-back transaction is not.

7 Security and observability

7.1 Enforcement

- **Sharing is declared, never inherited.** Apex REST classes run in system context unless told otherwise, so each endpoint class is explicitly `with sharing`.
- **CRUD/FLS enforced by the platform, not by hand.** All queries use `WITH USER_MODE`, and DML runs as `Database.insert(records, AccessLevel.USER_MODE)`. Hand-rolled `isAccessible()` checks drift the moment a field is added; user-mode enforcement does not.
- **No dynamic SOQL.** Every query is static with bind variables.
- **Ids are validated before use.** The path parameter goes through `Id.valueOf` inside a try/catch and is checked to be of type `Opportunity` before any query. A malformed Id is a clean `400`, not an unhandled `StringException`.
- **Not found and not permitted return the same `404`.** Distinguishing them turns the endpoint into an Id oracle.
- **Errors never leak internals.** The caller gets a typed code, a safe message and the correlation ID. The stack trace goes to the log.
- **Input is bounded.** Line counts, string lengths and numeric precision are validated before anything is queried.

D-10 · DESIGN DECISION — THE SYSTEM DEFAULTS TO INERT

Fields deployed through the Metadata API carry no field-level security for any profile, including System Administrator. Combined with user-mode enforcement everywhere, a freshly deployed org is not merely restricted but inert: even an admin cannot see `Active__c`.

The permission sets are therefore part of the deployable artefact, not a post-install click path, and assigning one is a required step rather than a footnote.

That is the correct failure mode. A system whose security defaults to open is one bad deployment away from an unprotected floor; this one defaults to closed and makes you say who gets in. Appendix A records how thoroughly this bit during the build — three separate times.

7.2 Least privilege, as actually established

`CPQ_Integration_User` grants read on the catalog objects, create/read/edit on the three objects it writes, read on `Product2 / Pricebook2 / PricebookEntry / Account`, read/edit on `Opportunity`, create/edit on `OpportunityLineItem`, Apex class access to the three endpoints, and exactly three user permissions: `ApiEnabled`, `ApexRestServices` and `EditOpplLineItemUnitPrice`. No Modify All Data, no View All.

Every one of those last three was added because the API failed without it, over HTTP, as the real principal. See Appendix A.

7.3 Logging

Logging is provided by **Nebula Logger**, an unlocked package, rather than by objects built for this project.

D-11 · DESIGN DECISION — LOGS MUST OUTLIVE THE TRANSACTION THEY DESCRIBE

A commit that fails rolls back to its savepoint — and a log row inserted in that transaction rolls back with it, so the failures you most want to see are exactly the ones that erase themselves.

Logs are therefore published as platform events, which commit independently of the enclosing transaction. Nebula's default save method is `EVENT_BUS` and its event publishes immediately, so this property is satisfied by the framework. It was verified directly against the org rather than assumed: an entry written inside a transaction that was then rolled back is still queryable afterwards.

`CpqLogger` remains as a thin adapter with the public surface it had before the framework was adopted, so no caller changed when the implementation did.

One field is added to Nebula's objects: `Correlation_Id__c`, the identifier shared with the React/NestJS platform. Nebula's own `TransactionId__c` identifies a Salesforce transaction; it cannot identify the caller's request, and one identifier spanning both systems is worth more than two good log stores that do not join. Everything else uses Nebula natively — endpoint as scenario, outcome and violation codes as tags, the Opportunity as the related record, exception details captured from the exception itself.

Violation codes are logged as tags deliberately: how far below floor reps are trying to go is the input to the next pricing conversation, and as tags "how often did `ADDON_BELOW_FLOOR` fire this quarter" is a report rather than a text search.

Monitoring. In production these events should be forwarded to the same observability stack Engineering already uses for the NestJS platform rather than building a second one inside Salesforce — one correlation ID spanning both sides is worth more than two dashboards that do not join.

8 Closed Won automation

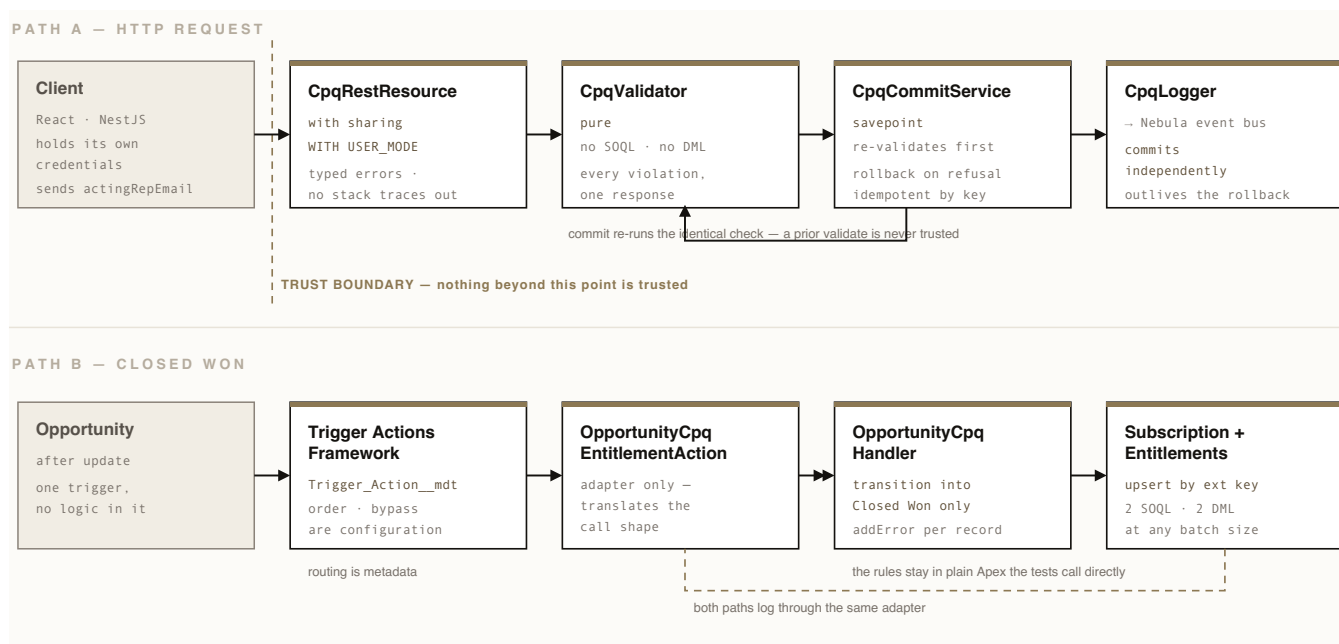


FIGURE 2 The two paths through the system. Path A is a request from the platform; the trust boundary marks where nothing is taken on faith, which is why commit re-validates rather than trusting a prior call. Path B is the automation a won deal triggers. Both log through the same adapter, and that log commits independently of whatever rolled back.

Apex, not a Record-Triggered Flow, as the brief requires.

D-12 · DESIGN DECISION — TRIGGER ROUTING IS CONFIGURATION

Which actions run, in what order, and whether they are bypassed are metadata records rather than lines of code — the same configuration-over-deployment principle the catalog and `Metering_Type__c` already follow.

With one action today this is close to even. It pays for itself against §9: the renewal and amendment actions become a class and a metadata record with an `Order__c`, rather than an edit to a trigger body that already has something in it. The bypass switch is also the mechanism a data migration needs, and that requirement always arrives eventually.

The handler fires only on the **transition into** Closed Won — `oldMap` stage is not won, new record `IsWon` — so re-saving a won Opportunity does nothing. Idempotency is enforced twice over: by that transition check, and structurally by `Subscription__c.External_Key__c` being a unique External Id upserted on the Opportunity Id. The transition check handles the common case; the unique key is what holds if a future entry point bypasses the trigger.

Bulk safety: queries and DML outside all loops, two SOQL and two DML regardless of batch size, correct for a 200-record Data Loader stage update. The test asserts limit consumption directly rather than record counts, because a per-record query is the kind of regression that passes a correctness test and fails in production at row 201.

Failure handling. Entitlements are created synchronously, so the subscription, the entitlements and the won stage commit together or not at all — a deal cannot be won carrying half-built entitlements. On failure the handler calls `addError` on that specific Opportunity, which fails that record only and lets the other 199 succeed, and logs through the event bus so the record survives the rollback. It does not swallow the exception.

If entitlement creation later needs a callout, it moves to a Queueable chained from the trigger and the guarantee weakens from transactional to eventually-consistent — worth doing deliberately, not by accident.

9 Deferred scope

Renewals, amendments, co-terming, proration and cancellation are the next phase, and the model is shaped so they are additions rather than a rewrite: entitlements already carry an active window and a `Superseded` status rather than being updated in place, subscriptions already carry term and end date, and committed lines already retain their catalog origin. An amendment becomes a new subscription superseding the old with a new entitlement generation, and the history of what a customer was entitled to *at a point in time* survives — which is what a billing dispute six months later actually needs.

Approval routing for sub-floor pricing is out of scope by instruction, but it is the most likely first change request once reps hit the floor. Because validation returns structured violations rather than throwing, routing a `PLAN_BELOW_FLOOR` to an approval path is a branch in the platform, not a redesign here.

10 Test strategy

10.1 Acceptance criteria as test methods

The full suite lives in the repository, and every `R-n` in §3 maps to a method named for the rule it asserts — the naming is the traceability, so there is no separate matrix to keep in step. The ones that carry the most weight:

- `validate_planPriceExactlyAtFloor_passes` and `validate_planPriceOneCentBelowFloor_fails` — the inclusive boundary, from both sides.
- `validate_subFloorPlanAndSubFloorAddOn_returnsBothViolations` — every violation in one response, not just the first.
- `commit_subFloorSkippingValidate_createsNothing` — the governing constraint, exercised the way a broken client would call it.
- `commit_secondCommit_replacesPriorCpqLinesOnly` — re-quoting replaces this API's own lines and leaves anything a human added alone.
- `commit_sameIdempotencyKeyTwice_createsOneSetAndReplaysResult` — the retry returns the original result rather than writing twice.
- `closedWon_brandPlusContacts500_entitlesOneThousandContacts` — the case the brief calls out.
- `closedWon_allInWithCompetitorAnalysisAddOn_entitlesOnceNotTwice` — the same reducer, with a binary feature arriving from both sides.
- `closedWon_bulk2000opportunities_withinGovernorLimits` — asserts limit consumption directly, not record counts, because a per-record query passes a correctness test and fails in production at row 201.

As built: 66 tests written for this project, all passing. Coverage on the project's own classes runs 78–100%. Note that `RunLocalTests` now executes roughly 1,480 tests, because the unlocked packages have no namespace and their tests count as local; the 66 is the honest number for what this project wrote.

10.2 The layer Apex tests cannot reach

Apex tests build a `HttpRequest` by hand and invoke the handler directly. Three things are therefore never exercised in CI:

- **urlMapping routing.** A typo passes all 66 tests and returns 404 in production.
- **Authentication.** No Apex test involves a Connected App, a token or an OAuth flow.
- **The integration user's real permissions.** Apex tests run as whoever runs them, which is an administrator.

A committed Postman collection closes that gap: ten requests, each asserting its own status and payload, run against a real org as `svc.revtech.cpq`. It also pins the JSON field names the client binds to, which typed deserialisation into DTO classes cannot catch — rename a DTO field and both sides move together while every external client breaks.

Everything in Appendix A was found this way, and none of it was visible to a passing test suite.

D-13 · DESIGN DECISION — TEST THE ADAPTER, NOT THE FRAMEWORK

The logging tests assert on the event `CpqLogger` builds, not the row `Nebula` derives from it. Persisting is `Nebula`'s responsibility and is covered by `Nebula`'s own suite; in Apex test context its handler does not run at all, so asserting on the row would be testing the framework's test harness. That the row genuinely survives a rolled-back transaction was verified against the org instead.

10.3 Running it

Deploy, permission sets, catalog seed, integration user and the test run are four commands and are listed in the repository README.

`scripts/demo.sh` runs the whole path over HTTP against a real org: catalog, a refusal at \$425, a pass at exactly \$450, a refused commit that writes nothing, an accepted commit, a retried commit that replays rather than double-writing, then Closed Won and the resulting entitlements. It closes on the overlap case from §5 — All In plus a Competitor Analysis the plan already includes — so the binary-versus-metered distinction is visible in the output rather than only in a test.

Appendix A What building it changed

Version 1.0 of this document was written before the system was called over HTTP by a least-privilege user. Everything below passed 66 Apex tests and failed the first real request. They are recorded rather than edited away, because the pattern matters more than any one of them: **an Apex test suite cannot see the boundary where an external caller actually arrives.**

F-1 — The Salesforce Integration licence cannot host this API. Version 1.0 specified one, on the reasoning that it is API-only and cheaper than a full seat. Probing object by object, that licence refuses `Read` on `Account`, `Opportunity` and `Product2`, and `Pricebook2` depends on `Product2`. Assigning `CPQ_Integration_User` to such a user fails outright. Because D-4 makes `PricebookEntryId` mandatory on every line, the licence can never work here. The principal now runs on a full Salesforce licence with the most restrictive profile available. The least-privilege argument survives; the cost argument does not. Raised as Q-5.

F-2 — `ApiEnabled` and `ApexRestServices` are not implied by anything else. Authentication succeeded and every endpoint returned `403 APEX_REST_SERVICES_DISABLED`. No `RestContext` test can catch this: the permission is checked at the HTTP boundary those tests never cross.

F-3 — A permission set that grants FLS only on its own custom fields is not functional. The standard fields the services read had no field permissions — `Opportunity.AccountId` above all. SOQL reports this as `No such column 'AccountId' on entity 'Opportunity'`, which reads like a typo and is actually a permission. `Account` object read is required too: a lookup to an object the running user cannot read does not exist as far as SOQL is concerned. Fields that are not FLS-controlled at all — `StageName`, `IsClosed`, `Quantity`, `UnitPrice`, `PricebookEntryId` — follow object access and must be *absent* from the permission set.

F-4 — `EditOpplLineItemUnitPrice` is what makes a CPQ possible at all. Without it, inserting a line below its `PricebookEntry` list price fails with *"User may not specify unit price different from list price"*. A system built entirely around negotiated pricing under a floor could not write a single discounted line. It is the least obvious requirement in the design and the one that most completely breaks it.

F-5 — Platform event fields cannot be External Ids. Adding `Correlation_Id__c` to `LogEntryEvent__e` as an External Id fails the deployment with a generic `UNKNOWN_EXCEPTION` that names nothing.

F-6 — Nebula does not copy subscriber custom fields from the event to the log entry. The mapping is an explicit `LoggerFieldMapping__mdt` record. Without it, `Correlation_Id__c` is populated on the event, the entry is written, and the column on `LogEntry__c` is silently null. Logging appears to work perfectly while the one field that joins these logs to the platform's is missing — the worst shape a defect can take.

F-1 through F-4 are all the same finding as **D-10**, arriving four more times. A system that defaults to closed tells you what it needs only when you ask it as the principal that will actually be asking.